

Week 12

A complete Data Science project

I choose a start-up business problem.

where early startup which as SaaS or a fintech app, doesn't having any historical data about frauds, like if that startup offers any subscriptions, fraudsters may use stolen credit cards(payment fraud), or if the company offers a promo code fraudster may create thousands of fake accounts to drain those promo codes.

My objective is to flag such a payment frauds accounts without harming the legit user and also ensuring the company's captial won't slip into hands of fraudsters.

*the dataset as decided.. the **PaySim Synthetic Dataset(from kaggle)**

```
In [2]: import pandas as pd

df = pd.read_csv('data/paysim_sample.csv')

print(df.shape)

df.head()
```

(318131, 11)

```
Out[2]:
```

	step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest
0	10	CASH_IN	367088.24	C291409265	1328.10	368416.34	C1132709076
1	10	PAYMENT	27004.49	C535489359	0.00	0.00	M2110652315
2	8	PAYMENT	2173.75	C927615422	820.00	0.00	M1658216730
3	10	TRANSFER	19670.92	C1562020231	5416.73	0.00	C2060926688
4	10	PAYMENT	17394.26	C1165434512	168543.14	151148.88	M490546299

About Dataset

step: unit of time. 1 step = 1 hour

type: CASH_IN, CASH_OUT, DEBT, PAYMENT and TRANSFER

amount: the charge or THE amount

nameOrig: customer who started the transaction

oldbalanceOrg: initial balance before the transaction

newbalanceOrig - new balance after the transaction.

nameDest - customer who is the recipient of the transaction

oldbalanceDest - initial balance recipient before the transaction. Note that there is not information for customers that start with M (Merchants).

newbalanceDest - new balance recipient after the transaction. Note that there is not information for customers that start with M (Merchants).

isFraud - This is the transactions made by the fraudulent agents inside the simulation. In this specific dataset the fraudulent behavior of the agents aims to profit by taking control or customers accounts and try to empty the funds by transferring to another account and then cashing out of the system.

isFlaggedFraud - The business model aims to control massive transfers from one account to another and flags illegal attempts. An illegal attempt in this dataset is an attempt to transfer more than 200,000 in a single transaction.

In [3]:

```
#-----structural Overview-----
print("-----DataFrame info-----")
print(df.info())

#-----Checking for NULL values---
print("\n-----Missing values-----")
print(df.isna().sum())
```

```
-----DataFrame info-----
<class 'pandas.DataFrame'>
RangeIndex: 318131 entries, 0 to 318130
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   step                  318131 non-null  int64
1   type                  318131 non-null  str
2   amount                318131 non-null  float64
3   nameOrig              318131 non-null  str
4   oldbalanceOrig        318131 non-null  float64
5   newbalanceOrig        318131 non-null  float64
6   nameDest              318131 non-null  str
7   oldbalanceDest        318131 non-null  float64
8   newbalanceDest        318131 non-null  float64
9   isFraud               318131 non-null  int64
10  isFlaggedFraud        318131 non-null  int64
dtypes: float64(5), int64(3), str(3)
memory usage: 35.3 MB
None
```

```
-----Missing values-----
step                0
type                0
amount              0
nameOrig            0
oldbalanceOrig      0
newbalanceOrig      0
nameDest            0
oldbalanceDest      0
newbalanceDest      0
isFraud             0
isFlaggedFraud      0
dtype: int64
```

```
In [4]: df1=df.copy(deep=True)
```

EDA

```
In [5]: print("--- FRAUD CLASS DISTRIBUTION ---")
print(df1['isFraud'].value_counts())
print("\n--- FRAUD PERCENTAGE ---")
print(df1['isFraud'].value_counts(normalize=True) * 100)
```

```
--- FRAUD CLASS DISTRIBUTION ---
```

```
isFraud
```

```
0    317735
```

```
1      396
```

```
Name: count, dtype: int64
```

```
--- FRAUD PERCENTAGE ---
```

```
isFraud
```

```
0    99.875523
```

```
1     0.124477
```

```
Name: proportion, dtype: float64
```

```
In [6]: print("--- TRANSACTION AMOUNTS: NORMAL VS FRAUD ---")
print(df1.groupby('isFraud')['amount'].describe())
```

```
--- TRANSACTION AMOUNTS: NORMAL VS FRAUD ---
```

	count	mean	std	min	25%	50% \
isFraud						
0	317735.0	1.771326e+05	5.614648e+05	0.16	13318.6000	74675.350
1	396.0	1.474271e+06	2.382247e+06	0.00	140689.1575	455223.335

	75%	max
isFraud		
0	208445.870	64234448.19
1	1407085.135	10000000.00

whoa.... a fruadster gain a max of 10M dollar... shock!!!

```
In [7]: print("--- FRAUD BY TRANSACTION TYPE ---")
# Filter only fraudulent transactions, then count the 'type'
print(df1[df1['isFraud'] == 1]['type'].value_counts())
```

```
--- FRAUD BY TRANSACTION TYPE ---
```

```
type
```

```
CASH_OUT    203
```

```
TRANSFER    193
```

```
Name: count, dtype: int64
```

only in the two type , fine fine!.

```
In [23]: # Keep only the transaction types where fraud actually happens
df2 = df1[df1['type'].isin(['TRANSFER', 'CASH_OUT'])].copy(deep=True)
```

```
In [28]: df_clean = df2.drop(columns=['nameOrig', 'nameDest', 'isFlaggedFraud'], errors='ignore')
df_clean
```

Out[28]:

	step	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	new
3	10	TRANSFER	19670.92	5416.73	0.00	63327.50	
9	9	CASH_OUT	282516.89	1376786.46	1094269.57	1048155.47	
10	8	TRANSFER	696571.67	0.00	0.00	17979799.39	
11	9	CASH_OUT	382856.16	6524.00	0.00	22118.00	
12	9	CASH_OUT	10169.19	0.00	0.00	991006.46	
...	...	...	...	...	...	...	...
318117	694	TRANSFER	61403.35	0.00	0.00	157981.13	
318124	707	CASH_OUT	23965.07	0.00	0.00	775215.12	
318125	688	CASH_OUT	82234.47	101920.00	19685.53	0.00	
318126	692	CASH_OUT	168885.86	22751.00	0.00	25353.76	
318129	690	TRANSFER	264801.24	30830.00	0.00	1813221.51	

138460 rows × 8 columns



In [29]:

```
# Binary encode the 'type' column
df_clean['type'] = df_clean['type'].map({'CASH_OUT': 0, 'TRANSFER': 1})
df_clean.head()
```

Out[29]:

	step	type	amount	oldbalanceOrg	newbalanceOrig	oldbalanceDest	newbalanceDe
3	10	1	19670.92	5416.73	0.00	63327.50	0.0
9	9	0	282516.89	1376786.46	1094269.57	1048155.47	1651625.7
10	8	1	696571.67	0.00	0.00	17979799.39	20448938.3
11	9	0	382856.16	6524.00	0.00	22118.00	0.0
12	9	0	10169.19	0.00	0.00	991006.46	1001175.6



In [34]:

```
df_clean.to_csv('data/cleaned_dataset_4_training.csv')
```

In [33]:

```
df_clean.shape
```

Out[33]:

```
(138460, 8)
```

In [41]:

```
from sklearn.model_selection import train_test_split

x= df_clean.drop('isFraud', axis=1)
y = df_clean['isFraud']

xtrain, xtest, ytrain, ytest = train_test_split(x,y,test_size=0.2, random_state=42, s

print(f'Training Fraud cases: {sum(ytrain)}')
print(f'Testing Fraud cases: {sum(ytest)}')
```

Training Fraud cases: 317

Testing Fraud cases: 79

Training a **xgboost** model

In [42]: `# !pip install xgboost`

scale_pos_weight: parameter to force the algorithm to care about that tiny 0.12% of fraud data by penalizing it heavily for missing them.

```
In [43]: import xgboost as xgb
from sklearn.metrics import confusion_matrix, classification_report

#calculating inbalance ratio for the scale_pos_weight(for xgboost model)
neg_cases = sum(ytrain == 0)
pos_cases = sum(ytrain == 1)
scale_weight = neg_cases / pos_cases
print(neg_cases, ',', pos_cases)
print(f'the value of the scale_pos_weight is: {scale_weight:.2f}')
```

110451 , 317

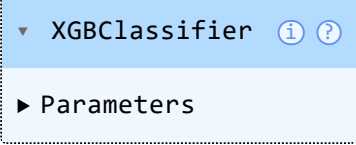
the value of the scale_pos_weight is: 348.43

```
In [44]: #creating the xgboost model
xgb_model = xgb.XGBClassifier(
    scale_pos_weight = scale_weight,
    random_state = 42,
    eval_metric = 'aucpr',
    use_label_encoder = False
)
```

```
In [45]: #training the model
xgb_model.fit(xtrain,ytrain)
```

c:\Users\Mohith M\AppData\Local\Programs\Python\Python314\Lib\site-packages\xgboost\training.py:200: UserWarning: [12:48:52] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:782:
Parameters: { "use_label_encoder" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

Out[45]: 

```
In [46]: #testing and printing the classification_report and confusion matrix
ypred = xgb_model.predict(xtest)

print("-----CONFUSION MATIX-----")
print(confusion_matrix(ytest, ypred))

print('-----CLASSIFICATION REPORT-----')
print(classification_report(ytest, ypred))
```

```

-----CONFUSION MATIX-----
[[27603    10]
 [   10   69]]
----CLASSIFICATION REPORT----

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	27613
1	0.87	0.87	0.87	79
accuracy			1.00	27692
macro avg	0.94	0.94	0.94	27692
weighted avg	1.00	1.00	1.00	27692

Reading confusion matrix:

TP: 69: outof 79 model predicted 69 transaction as fraud transaction

FN: 10: (bottom-left): model slipped 10 fraudster.

TN: 10: (top-right): model flagged 10 innocent transaction as fraud.

```

In [47]: import joblib
import os

# Create a deployment folder if you haven't already
os.makedirs('deployment', exist_ok=True)

# Save the trained XGBoost model to the deployment folder
model_filename = 'deployment/xgboost_fraud_model.joblib'
joblib.dump(xgb_model, model_filename)

print(f"Success! Model securely saved to: {model_filename}")

```

Success! Model securely saved to: deployment/xgboost_fraud_model.joblib